List 2: Tokenization & Embeddings

Labs: Mon 19 Oct, Mon 26 Oct 2026 · Due: Sun 8 Nov 2026, 23:59 · 1 point

- For each task, show the requested numbers or plots and explain them in two to four sentences. The list is graded as a whole, in steps of 0.25, by how complete, correct and well explained your solution is.
- Data, code structure and names are up to you; defaults are given for when you have no preference. Keep the signatures of the two given definitions (`create_dataloader` , `GPTEmbedding`), because later lists reuse them (check them with Materials, Appendix A).
- Background and the Python and PyTorch functions you need for Task X.Y are in Materials X.Y.
- Submit this notebook run from top to bottom with outputs, or Python scripts with a PDF report of at most four pages (results and explanations per task). Late: –10% per day, 0 after 5 days.
- Tasks marked *(optional)* are not graded: the full point is available from the other tasks. They are there for depth and for project ideas. In exceptional cases the grader may take them into account, within the 1-point maximum.
- Individual work. Name the AI tools you used at the top of your solution.

```
import tiktoken
import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.data import DataLoader, Dataset
```

Lab 3

3.1 A word tokenizer

Default text: `the-verdict.txt` (as in List 1). Tasks 3.2–3.5 use its first 90% for training and its last 10% as held-out text.

- Split the text with a regular expression that separates punctuation and whitespace, build the vocabulary from the whole text, and implement `encode` and `decode` . Reference: the regular expression of Materials 3.1 gives 1130 distinct tokens.
- Find a passage of the text for which `decode(encode(text)) != text` and explain which information the split discarded. Repair the spacing before punctuation in `decode` and say what still cannot be recovered.

3.2 Unknown words and document boundaries

- Add `<|unk|>` and `<|endoftext|>` to the vocabulary (1132 entries) and encode two short documents joined by `<|endoftext|>` , one of them with words that are not in the text.

- Build the vocabulary from the first 90% of the text only and report the out-of-vocabulary (OOV) rate of the last 10%: the share of its tokens that become `<|unk|>`. Reference: 51 of 517 tokens (9.9%). Explain why `<|unk|>` breaks the round trip.

3.3 BPE on letters

Default: the English words of the default text.

- Split the first 90% of the text into chunks with `re.findall(r" ?\w+| ?[^\w\s]+|\s+", text)`: a word with its leading space, a run of punctuation, or whitespace (Materials 3.3). Implement BPE training with the characters of the training text as the base vocabulary (IDs in sorted order; ties broken by the smaller pair of IDs; no merges across chunks), `encode` (apply the learned merges by rank inside each chunk) and `decode`. Reference: with `low`, `lower` and `lowest` as three chunks, the first three merges are `l+o`, `lo+w` and `low+e`, as in Lecture 3.
- Train with 256 merges and plot the length of the training text in tokens against the number of merges; print the first merges and the longest tokens. Reference: 62 distinct characters; the 18,431 characters become 8,388 tokens; the first merges are `␣t`, `he`, `␣a`.
- Check `decode(encode(s)) == s` on the held-out text. Then encode `Łódź` or a Chinese word and explain what fails.

3.4 From letters to bytes: byte-level BPE

- Replace the base vocabulary by the 256 UTF-8 byte values (merged tokens get IDs 256, 257, ...; ties broken by the smaller pair of IDs); `decode` concatenates the bytes and then decodes UTF-8. Train on the same chunks with 256 merges (vocabulary size 512). Show that on this ASCII text the merges are the same as in Task 3.3, and that the round trip now works for any string, including `Łódź` and Chinese.
- Train your byte-level BPE with about 60 merges on a short text full of multi-byte characters, for example a few Polish and Chinese sentences of your choice. Classify each new token as ASCII, whole multi-byte character(s), part of one character, or crossing a character boundary, and show which characters end up as one token and which stay split into bytes. Relate what you see to Lecture 3, Q&A, Question 1.
- Compare your tokenizer with GPT-2's (`tiktoken`) on the held-out text and on a short Polish text: bytes per token, and whether `decode(encode(s)) == s`. Explain the difference between the two languages. Reference: GPT-2 encodes the whole text in 5145 tokens; on the held-out text your tokenizer gives about 2.3 bytes per token and GPT-2 about 3.8.
- Find a GPT-2 token whose bytes are not valid UTF-8 on their own, and show what decoding token by token does to it.
- Train on the training text as one chunk, without the split, and compare the learned merges and the compression. *(optional)*

3.5 Comparing tokenizers with bits per byte

- Fit the add-α bigram model of List 1, Task 2.3, to token IDs instead of characters, for three tokenizers: bytes ($V = 256$) and your byte-level BPE with $V = 512$ and $V = 1024$ (default $\alpha = 0.1$). Fit on the first 90% and report, for each tokenizer, the per-token perplexity of the held-out text and its bits per byte $\mathrm{BPB} = N\bar{\ell}/(M \ln 2)$: N held-out tokens with mean negative log-likelihood $\bar{\ell}$ (in nats), M UTF-8 bytes of the held-out text.
- Explain why the two measures rank the tokenizers in opposite orders. Reference: perplexity about 11.5, 87 and 290; BPB about 3.53, 2.85 and 2.83.

3.6 Tokenization consequences *(optional)*

- With `tiktoken`, show how GPT-2 splits `strawberry` with and without a leading space, numbers such as `12345` and `1234 + 5678`, and repeated spaces. Relate what you see to the difficulties LLMs have with counting letters and with arithmetic.

- **Checkpoint:** your byte-level BPE (Task 3.4) gives `decode(encode(s)) == s` for every string in Materials, Appendix B (empty text, leading and repeated spaces, newlines, Polish, Japanese, an emoji with a skin-tone modifier, and the literal text `<|endoftext|>`).

Lab 4

4.1 An embedding is a one-hot vector times a matrix

- Show numerically that `nn.Embedding(V, d)` gives the same outputs as a one-hot input times `nn.Linear(V, d, bias=False)` whose weight is the transposed embedding matrix, and the same gradients for the same loss. Show that only the rows of the tokens that were looked up get non-zero gradients.
- Time both for $V = 50257$, $d = 768$ and 256 token IDs. *(optional)*

4.2 Sliding-window samples

- Implement `create_dataloader`. Derive the number of windows for N tokens, window length L (`max_length`) and stride s, and check it against `len(loader.dataset)` for several (L, s). Reference: the whole text with $L = s = 4$ gives 1286 windows.
- Print the first two input and target windows for stride 1 and for stride L, as tokens. Explain the overlap and why stride L is a common default.

```python
def create_dataloader(
    text: str,
    batch_size: int = 4,
    max_length: int = 256,
    stride: int = 128,
    shuffle: bool = True,
    drop_last: bool = True,
    num_workers: int = 0,
) -> DataLoader:
    """Next-token (inputs, targets) windows of GPT-2 token IDs.

    Batches have shape (batch_size, max_length); see Materials, Appendix A.
```

```
    """
    raise NotImplementedError
```

4.3 Token and positional embeddings

- Implement `GPTEmbedding`. Report its number of parameters for the GPT-2 small configuration (vocabulary 50257, context length 1024, $d = 768$) with `count_parameters` from List 1, and check it by hand. Reference: 39,383,808.
- Show that without the positional term, permuting the input tokens permutes the output vectors in the same way, and that with it the same token at two positions gets two different vectors.

```
class GPTEmbedding(nn.Module):
    """Token embedding plus learned absolute position embedding."""

    def __init__(self, vocab_size: int, context_length: int, emb_dim: int):
        super().__init__()
        raise NotImplementedError

    def forward(self, idx: torch.Tensor) -> torch.Tensor:
        """(B, T) token IDs with T <= context_length -> (B, T, emb_dim)."""
        raise NotImplementedError
```

4.4 Sinusoidal positional encodings

- Implement the sinusoidal encoding of Vaswani et al. (Materials 4.4) and show it as a heatmap for $T = 100$ positions and $d = 64$.
- Plot $\mathrm{PE}(t) \cdot \mathrm{PE}(t + k)$ against $k = 0, \ldots, 50$ for several t. Explain why the curves coincide and why they start at $d/2$.
- Build the block-diagonal matrix $R(k)$ and check in `float64` that $\mathrm{PE}(t + k) = R(k)\,\mathrm{PE}(t)$ for all t and several k.

4.5 Skip-gram with negative sampling *(optional)*

- Train skip-gram with negative sampling (Materials 4.5) on the words of the default text and print the nearest neighbours, by cosine similarity, of a few frequent words. Comment on what a text of a few thousand words can and cannot teach.

4.6 Your own experiment *(optional)*

About one hour of work: one question this list raised, answered with one plot or table and a paragraph. Ideas are in Materials 4.6.

- **Checkpoint:** a batch from `create_dataloader(text, batch_size=8, max_length=4, stride=4, shuffle=False)` passed through `GPTEmbedding(50257, 4, 256)` has shape `(8, 4, 256)`, and its targets are its inputs shifted by one (code in Materials, Appendix B).